

Paper B-11: Computational Architecture: A Constraint-Driven Flux Dynamics (CDFD) Perspective

Steve Bico Mujjabi, MD
Independent Researcher
ORCID: 0009-0001-0556-5516

May 2026

Abstract

This paper asks whether Constraint-Driven Flux Dynamics (CDFS) and Adaptive Flux Limitation (AFL) can give the named system a sharper audit language. The working variables are driving flux Φ , constraint C , surface responsiveness S , and structural memory M_s . The useful question is plain: what can be measured, what would count as overload, and what result would make the mapping fail? In this paper we apply Adaptive Flux Limitation (AFL) to computer architecture, modelling instruction throughput as constrained transport governed by the potential Φ (instruction retirement rate, FLOP/s), with multi-dimensional constraint $C = (C_{mem}, C_{pwr}, C_{IPC})$ encoding memory bandwidth, thermal power budget, and instruction-level parallelism limits. The effective $\Psi_s^* = J/C_{eff}$ where $C_{eff} = \min(C_{mem}, C_{pwr}, C_{IPC})$ classifies processor states: memory-bound, power-bound, or IPC-bound.

We model the hypothesis that roofline performance models, cache hierarchy design, heterogeneous chip architecture (CPUs + GPUs + NPUs), and dark-silicon management emerge naturally from AFL multi-wall dynamics. Dennard scaling failure is recast as C_{pwr} wall formation; modern chiplet design is AFL constraint diversification.

Status: Inferred from AFL first principles; quantitative predictions are hypothesis-level.

Series tag: Eng-B11

Greek-symbol convention. Unless a manuscript states a tighter equation-local meaning, Greek symbols are local CDFS variables or coefficients, not universal constants. Here Φ denotes driving flux, Ψ_s denotes the adaptive operating ratio, Ω denotes a domain or state space, and Λ denotes the Life Number where used. The coefficients α , β , and γ denote accumulation or reinforcement, relaxation or decay, and diffusion, coupling, or redistribution. The symbols δ and ϵ denote perturbation or tolerance scales, while η , λ , μ , ν , ρ , σ , τ , and ϕ denote local efficiency, rate, baseline, frequency, density or correlation, noise or variance, time-scale, and phase or field terms. Subscripts restrict any coefficient to the named pathway, tissue, domain, or experiment.

1 Series Context

Part I sets the flow-constraint language used here [1]. Part II develops the Life Number and the origins-of-life use of the same notation [2]. Part III applies AFL to biology and medicine [3]. Part IV is the cross-domain stress test: it asks where the notation clarifies measurement, and where it becomes only a metaphor.

2 Introduction

2.1 The Mujjabi Universal Capacity Law

Building directly upon the Fundamental Physics (Part I) [1], the **Mujjabi Universal Capacity Law** proposes that across all scales—from black hole event horizons to global financial markets and power grids—driving high flux (Φ) against a rigid topological constraint (C) can drive system saturation. When Φ/C approaches critical thresholds, the system does not scale linearly; it undergoes a topological phase transition.

2.2 The Mujjabi Universal Memory Law

The **Mujjabi Universal Memory Law** frames persistent systemic collapse. When a complex network fails to clear its structural memory (M_s) and its relaxation parameter decays ($\tau_{relax} \rightarrow \infty$),

temporary stressors become permanent rigidities. This is the proposed CDFD mechanism behind ecological death, economic depressions, and infrastructural gridlock.

Modern processors contain $> 10^{10}$ transistors operating at $> 10^9$ Hz, yet practical throughput is limited by three architectural constraints that have converged to form permanent performance ceilings. These “walls” are among the most consequential engineering challenges in computer science:

1. **Memory wall:** DRAM bandwidth grows at $\sim 26\%$ /year while CPU performance grew at $\sim 52\%$ /year (through 2004), creating an ever-widening gap;
2. **Power wall:** Dennard scaling broke down around 2005; frequency can no longer increase without thermal runaway (“dark silicon”);
3. **IPC wall:** instruction-level parallelism (ILP) extractable by out-of-order execution saturates at $\sim 4\text{--}6$ IPC for most workloads.

AFL unifies these three walls as a multi-dimensional constraint system: processor throughput J is jointly bounded by three independent C values, and the effective constraint is $C_{eff} = \min(C_{mem}, C_{pwr}, C_{IPC})$ — the weakest wall determines performance. This provides a single $\Psi_s^* = J/C_{eff}$ diagnostic for architectural bottleneck ($\Psi_s > 1$ lock state) identification.

3 Core Hypothesis

Computational throughput in processor architecture is governed by a multi-dimensional AFL system in which three converging constraints (memory bandwidth, power budget, instruction parallelism) jointly bound instruction retirement rate, with heterogeneous chip design representing the AFL strategy for constraint diversification across varied workload types.

AFL architecture hypothesis: the roofline performance model is an AFL two-constraint model (C_{mem} and $C_{peak-FLOP}$), and the “roofline knee” is the AFL $\Psi_s^* = 1$ operating point where compute and memory constraints are balanced. Modern performance engineering seeks workloads where $\Psi_s^* \approx 1$ on the compute roofline rather than the memory roofline. **Status: Hypothesis; roofline model provides direct structural correspondence.**

4 Computational Potential Field

We define the generalised processor potential:

$$\Phi = \text{instruction retirement rate} = \text{IPC} \times f \quad (1)$$

where IPC is instructions per cycle and f is clock frequency. The AFL transport equation governs how Φ flows through the processor pipeline:

$$J = -\frac{1}{C_{eff}} \nabla \Phi \quad (2)$$

where C_{eff} encodes the binding constraint at any given workload.

5 The Three-Wall AFL System

5.1 Memory Wall: C_{mem}

Memory bandwidth constraint:

$$C_{mem} = \frac{\text{Arithmetic Intensity}}{\text{Peak Memory Bandwidth}} = \frac{FLOP/byte}{BW_{peak}}. \quad (3)$$

A workload is memory-bound when $J_{compute} > C_{mem}$: the processor can perform more operations than the memory subsystem can supply data for. AFL prediction: memory-bound workloads have $\Psi_{s,mem} > 1$; they waste compute cycles waiting for data.

Cache hierarchy is AFL constraint reduction: L1 cache reduces effective C_{mem} for data with high temporal locality by a factor of ~ 100 (DRAM vs. L1 latency ratio).

5.2 Power Wall: C_{pwr}

Thermal power constraint:

$$C_{pwr} = \frac{P_{TDP}}{P_{per-transistor}} = \frac{P_{TDP}}{\alpha C V^2 f}, \quad (4)$$

where P_{TDP} is thermal design power, C (here capacitance) is switching capacitance per transistor, V is supply voltage, f is frequency.

Dennard scaling: through 2004, $V \propto f$ kept $P_{per-transistor} \propto f^{-2}$, so scaling f preserved P_{total} . When V could not decrease further (leakage current), $P_{pwr} \propto f^3$ and scaling ceased. AFL interpretation: C_{pwr} wall formed when Dennard scaling exhausted the adaptive mechanism ($\beta_{Dennard} \rightarrow 0$).

Dark silicon: when only a fraction f_{active} of transistors can be active simultaneously at design power, $J_{compute} = f_{active} \cdot J_{peak}$. AFL $\Psi_{s,pwr} = J_{peak}/(f_{active} C_{pwr})$: dark silicon is the AFL response to power-wall constraint — reduce J_{active} to maintain $\Psi_{s,pwr} \leq 1$.

5.3 IPC Wall: C_{IPC}

Instruction-level parallelism constraint:

$$C_{IPC} = \frac{1}{ILP_{extractable}} \quad (5)$$

where $ILP_{extractable}$ is the average instruction-level parallelism available in the instruction stream. Out-of-order execution, branch prediction, and speculative execution increase $ILP_{extractable}$, reducing C_{IPC} .

Amdahl's law at the instruction level: sequential data dependencies (RAW hazards) impose irreducible C_{IPC} . Modern OOO processors extract ~ 4 – 6 IPC for integer workloads; the IPC wall is the AFL sequential-fraction constraint at microarchitectural scale.

6 Conservation Dynamics

Processor throughput obeys conservation:

$$\frac{\partial \Phi}{\partial t} + \nabla \cdot J = S \quad (6)$$

where S represents instruction supply (from the instruction cache and decoder) and J represents instruction retirement (completed execution). The pipeline is the AFL transport medium; the reorder buffer (ROB) is the AFL queue.

7 Adaptive Constraint Dynamics

Architectural constraints evolve dynamically:

$$\frac{dC_{eff}}{dt} = \alpha|J| - \beta C_{eff} \quad (7)$$

where α governs workload-induced constraint stress (high throughput fills ROB, causes branch mis-predictions, increases cache pressure) and β governs microarchitectural recovery (branch predictor warms up, prefetcher learns access patterns, TLB reloads).

Physical mechanisms:

- **Branch misprediction:** wrong-path speculation fills ROB; recovery flushes pipeline (transient $J \rightarrow 0$, C spikes);
- **Cache miss cascade:** L1 miss $\rightarrow$ L2 miss $\rightarrow$ L3 miss $\rightarrow$ DRAM: each level increases effective C_{mem} by $\sim 10\times$;
- **TLB miss:** virtual-to-physical address translation miss requires page-walk, adding ~ 100 cycle penalty to effective C ;
- **Hardware prefetcher:** learns memory access patterns, reducing effective C_{mem} by prefetching data before it is needed (β mechanism).

7.1 Adaptive Surface Dynamics: Memory and Responsiveness

The operating ratio Ψ_s^* is mediated by adaptive surface components:

$$\Psi_s^* = (\Phi/C) \cdot S \cdot M_s \quad (8)$$

where S (Surface Responsiveness) and M_s (Surface Memory) evolve as:

$$\frac{dS}{dt} = \kappa_s(\Psi_s^* - S) \quad (9)$$

$$\frac{dM_s}{dt} = \Phi S - d_m M_s \quad (10)$$

These components represent the homeostatic capacity of the system to regulate its own operating ratio through memory and responsiveness.

8 Flux-Constraint Number

$$\Psi_s^* = \frac{J}{C_{eff}} = \frac{J}{\min(C_{mem}, C_{pwr}, C_{IPC})} \quad (11)$$

8.1 Memory-Bound Regime (C_{mem} dominates)

Workload has low arithmetic intensity; processor waits for data. Memory-bound workloads: streaming operations (memcpy, matrix transpose), sparse linear algebra, large graph traversal. AFL $\Psi_{s,mem} > 1$: processors can compute faster than memory can supply.

8.2 Compute-Bound Regime (C_{IPC} or C_{pwr} dominates)

Workload has high arithmetic intensity; memory latency is hidden. Dense matrix multiplication (GEMM), FFT on cached data, neural network forward pass on GPU. AFL $\Psi_{s,mem} < 1$: memory is not the bottleneck ($\Psi_s > 1$ lock state); compute is the limit.

8.3 Thermally Limited Regime (C_{pwr} dominates)

All transistors operational but power budget exhausted; frequency must be reduced below rated speed. Mobile processors and sustained HPC workloads operate in this regime. AFL $\Psi_{s,pwr} \rightarrow 1$: processor executes at thermal steady-state.

9 The Roofline Model as Two-Wall AFL

The Roofline performance model (Williams, Waterman, Patterson 2009) plots:

$$\text{Attainable GFLOP/s} = \min(P_{peak}, B_{peak} \times I) \quad (12)$$

where I is arithmetic intensity (FLOP/byte), P_{peak} is peak compute, B_{peak} is peak bandwidth. This is the AFL two-wall model:

$$J = \min\left(\frac{\Phi}{C_{mem}}, \frac{\Phi}{C_{IPC}}\right) \quad (13)$$

$$\Psi_s^* = \frac{J}{\min(C_{mem}, C_{IPC})} \quad (14)$$

The roofline “knee” at $I_{knee} = P_{peak}/B_{peak}$ is the AFL $\Psi_s^* = 1$ balanced point. Optimising code for the roofline model is AFL constraint balancing: raise I (increase compute per byte) until $\Psi_{s,mem} = \Psi_{s,IPC} = 1$.

10 Cache Hierarchy as AFL Constraint Reduction

Cache hierarchy reduces effective C_{mem} :

$$C_{mem,eff} = h_1 C_{L1} + (1 - h_1) h_2 C_{L2} + (1 - h_1)(1 - h_2) C_{DRAM} \quad (15)$$

where h_1, h_2 are L1, L2 hit rates and $C_{L1} \ll C_{L2} \ll C_{DRAM}$.

AFL cache design principle: keep C_{L1} small enough that $\Psi_{s,L1} < 1$ for the temporal-locality working set; C_{L2} for the spatial-locality working set. Cache size is AFL constraint buffer sizing.

11 Heterogeneous Architecture as AFL Constraint Diversification

Modern chips combine multiple processor types:

$$\text{CPU} \rightarrow \text{low-latency sequential workloads} \quad (16)$$

$$\text{GPU} \rightarrow \text{high-throughput parallel workloads} \quad (17)$$

$$\text{NPU/TPU} \rightarrow \text{matrix-multiplication workloads} \quad (18)$$

This is AFL constraint diversification: each accelerator has different $(C_{mem}, C_{pwr}, C_{IPC})$, and the heterogeneous system routes workloads to the accelerator where $\Psi_s^* \approx 1$:

$$\text{optimal routing: } J_{workload} \rightarrow \text{accelerator}^* = \arg \min_k \Psi_{s,k} \quad (19)$$

GPUs have $C_{IPC,GPU} \ll C_{IPC,CPU}$ (massively parallel), but $C_{mem,GPU} \sim C_{mem,CPU}$ per core. Matrix operations with high arithmetic intensity ($I > 100$ FLOP/byte) have $\Psi_{s,GPU} \approx 1$ (compute-bound on GPU) while $\Psi_{s,CPU} \gg 1$ (memory-bound on CPU).

12 Chiplet Architecture

Modern chiplet design (AMD EPYC, Apple M-series, Intel Meteor Lake) disaggregates monolithic die into compute tiles, memory tiles, and I/O tiles. This is AFL modularisation:

$$C_{eff,chiplet} < C_{eff,monolithic} \quad (20)$$

because each chiplet is optimised for its constraint type, and inter-chiplet interconnects (UCIe, HBM stacking) are designed for the specific J between chiplet types.

13 Spatial Self-Organization

Processor microarchitecture self-organises into execution hierarchies (pipeline stages), throughput corridors (critical paths), caching hubs (shared L3), and adaptive resource pools (ROB, issue queue). This organisation minimises effective execution constraint while maximising instruction retirement throughput.

AFL-PDE simulations (`supplementary_eng.11.py`) model three-wall AFL: $\Psi_{s,mem}$, $\Psi_{s,pwr}$, $\Psi_{s,IPC}$ evolve under a synthetic workload trace, showing which wall is binding at each time step and how microarchitectural optimisations shift the binding constraint.

14 Predictions

1. **Roofline knee:** optimal arithmetic intensity $I^* = P_{peak}/B_{peak}$; AFL confirms this is the $\Psi_{s,mem} = \Psi_{s,IPC}$ balance point.
2. **Dark silicon fraction:** fraction of active transistors $f_{active} = P_{TDP}/(\alpha_{sw}CV^2fN)$; AFL predicts f_{active} scales inversely with frequency.
3. **Cache sizing:** optimal L1 cache size equals temporal working-set size; AFL predicts $h_1 = 1 - C_{L2}/C_{DRAM}$ at optimal sizing.
4. **Heterogeneous routing:** workloads with $I > I_{GPU}^*$ should route to GPU; AFL predicts routing threshold from accelerator roofline parameters.
5. **Chiplet benefit:** chiplet disaggregation reduces C_{eff} by $\delta C \propto N_{tiles} \cdot \Delta C_{specialisation}$; AFL quantifies chiplet advantage.

Status: Predictions 1–2 consistent with established computer architecture results; predictions 3–5 are AFL extensions requiring microarchitectural measurement.

15 Computational Interpretation

The companion script `supplementary_eng_11.py` implements AFL three-wall architecture:

- workload trace with varying arithmetic intensity $I(t)$;
- three constraint channels: $C_{mem}(I, BW)$, $C_{pwr}(f, V, N)$, $C_{IPC}(ILP)$;
- $\Psi_{s,eff} = J / \min(C_{mem}, C_{pwr}, C_{IPC})$ plotted vs. time;
- roofline: performance vs. intensity curve with memory and compute bounds;
- outputs: three-wall Ψ_s^* timeline, roofline plot, cache sizing recommendation.

16 Discussion

AFL unifies computer architecture under a three-wall multi-constraint framework. The memory wall, power wall, and IPC wall are three independent AFL constraints; the roofline model is two-wall AFL; heterogeneous architecture is AFL constraint diversification; chiplets are AFL modularisation. The AFL architectural design principle: minimise $C_{eff} = \min(C_{mem}, C_{pwr}, C_{IPC})$ by balancing across all three constraints simultaneously.

The key AFL insight: architectural optimisation should target $\Psi_{s,mem} = \Psi_{s,pwr} = \Psi_{s,IPC} = 1$ simultaneously (all walls equally binding), which maximises efficient utilisation of all resources. Modern heterogeneous designs approach this ideal by routing each workload type to the accelerator where its specific wall is the binding constraint.

17 Measurement Layer

The first job in any domain is to choose proxies before drawing conclusions. Φ may be a rate of material, energy, information, capital, or signal flow. C is the bottleneck that slows or redirects that flow. S measures the system's ability to reconfigure under stress, and M_s records the past load that still changes present behavior. A useful application gives those four terms observable definitions, a time scale, and a failure condition. If a standard domain model explains the same data more cleanly, the CDFD/AFL mapping should give way.

17.1 Current Runtime Diagnostic: Universal Network Cascade

The diagnostic script `Part_E_Synthesis/supplementary/run_partiv_discovery.py` keeps this paper tied to the current CDFD Runtime [4], including RK4 field updates, surface dynamics, structural-memory diffusion, and a finite-value audit. In the May 2026 run, the localized hub-drive stress test ended with hub $\Psi_s = 14.8$, peak network $\Psi_s = 14.8$, 21 nodes above $\Psi_s > 1.5$, and 37 maximum memory-locked nodes. I treat those numbers as a runtime sanity check and as a target for later domain measurement, not as a substitute for field data.

18 Conclusion

Computer architecture evolves through constrained instruction-transport dynamics governed by three converging AFL constraints: memory bandwidth, power budget, and instruction parallelism. AFL provides a unified framework describing the roofline model, dark silicon, cache hierarchy,

heterogeneous design, and chiplet architecture within a single multi- C AFL system, connecting computational architecture science to all other Part IV Engineered papers through the universal AFL equation.

References

- [1] Steve Bico Mujjabi. Cdfd part i: Fundamental physics - constraint-driven field theory and flux dynamics, 2026.
- [2] Steve Bico Mujjabi. Cdfd part ii: Origins of life and tri-regime bioenergetics - constraint-driven flux dynamics, 2026.
- [3] Steve Bico Mujjabi. Cdfd part iii: Adaptive flux limitation biology and medicine - constraint-driven flux dynamics, 2026.
- [4] Steve Bico Mujjabi. Cdfd runtime: Constraint-driven flux dynamics and cdf execution engine, 2026.